Aula 02 — Variáveis, tipos de dados e primeiros cálculos

Guardando informações, recebendo dados e mostrando resultados em Python

Dieimes Nunes

Retomando a Aula 01

Na aula anterior, vimos que programar não começa digitando código.

Programar começa quando conseguimos:

1. entender o problema;
2. organizar os passos;
3. identificar entradas;
4. pensar no processamento;
5. apresentar uma saída.

No final da aula anterior, ficou uma pergunta:

Como fazer o Python calcular o valor total de uma compra?

Hoje vamos avançar justamente nessa direção: **variáveis**, **tipos de dados**, **entrada**, **saída**, **conversão** e **primeiros cálculos**.

Antes de programar: uma situação simples

Imagine um sistema de cadastro de aluno.

Ele precisa guardar algumas informações:

- nome;
- idade;
- cidade;
- curso;
- telefone;
- e-mail.

Pergunta para a turma:

> Como o computador consegue guardar essas informações enquanto o programa está funcionando?

Variável: uma primeira ideia

Uma **variável** é um espaço usado para guardar uma informação durante a execução do programa.

Podemos pensar em uma variável como uma **caixinha com nome**.

Exemplo:

```
nome = "Ana"
idade = 16
cidade = "Ivaiporã"
```

Cada variável possui:

Exemplo da caixinha

Imagine três caixinhas:

```
nome    → Ana
idade   → 16
cidade  → Ivaiporã
```

Quando o programa precisar mostrar o nome, ele procura a caixinha chamada `nome`.

Quando precisar usar a idade, ele procura a caixinha chamada `idade`.

O nome da variável ajuda o programador a entender o que está sendo guardado.

Atividade 1 — Identificando variáveis

No caderno, observe a situação abaixo.

Situação: um sistema simples para cadastrar um produto de mercado.

O sistema precisa guardar:

- nome do produto;
- preço;
- quantidade em estoque;
- categoria;
- validade.

Responda:

1. Quais poderiam ser os nomes das variáveis?
2. Quais informações seriam textos?
3. Quais informações seriam números?
4. Alguma informação poderia ser usada em cálculo?

Possível resposta da Atividade 1

Uma possível organização seria:

```
nome_produto      → texto
preco             → número real
quantidade        → número inteiro
categoria         → texto
validade          → texto ou data
```

Em Python, poderíamos começar assim:

```python
nome_produto = "Arroz"
preco = 24.90
quantidade = 15
categoria = "Alimentos"
validade = "10/12/2026"
```

Nomeando variáveis

O nome da variável deve ajudar na leitura do programa.

Evite nomes sem significado:

```
x = "João"
y = 17
z = 8.5
```

Prefira nomes claros:

```
nome_aluno = "João"
idade_aluno = 17
media_aluno = 8.5
```

O código precisa funcionar, mas também precisa ser entendido por pessoas.

Regras básicas para nomes de variáveis

Em Python, o nome da variável:

- pode ter letras;
- pode ter números;
- pode usar underline _;
- não pode começar com número;
- não pode ter espaço;
- não deve usar acentos;
- não deve ser uma palavra reservada da linguagem.

Exemplos válidos:

```
nome
idade
nota1
media_final
preco_produto
```

Exemplos que devem ser evitados

```
2nota = 8
nome completo = "Ana Maria"
preço = 10.50
class = "Turma A"
```

Problemas:

- `2nota` começa com número;
- `nome completo` tem espaço;
- `preço` tem acento;
- `class` é uma palavra reservada.

Melhor:

```
nota2 = 8
nome_completo = "Ana Maria"
```

```
preco = 10.50
turma = "Turma A"
```

Atribuição

Atribuição é o ato de colocar um valor dentro de uma variável.

Em Python usamos o sinal de igual `=`.

```
idade = 18
```

Leia assim:

> A variável idade recebe o valor 18.

Não leia como uma igualdade matemática comum.

Em programação, o lado direito é calculado primeiro e depois guardado no lado esquerdo.

Exemplo de troca de valor

Uma variável pode receber um novo valor durante o programa.

```
idade = 18
print(idade)

idade = 19
print(idade)
```

A variável continua com o mesmo nome, mas o valor foi alterado.

Isso é muito comum em programação.

Exemplo com contador

```
contador = 0
contador = contador + 1
contador = contador + 1
contador = contador + 1

print(contador)
```

Pergunta:

> Qual valor será mostrado no final?

A variável começa com `0` e vai aumentando de um em um.

Esse conceito será muito importante quando estudarmos repetições.

Atividade 2 — Teste de mesa rápido

Sem executar no computador, descubra o valor final das variáveis.

```
numero = 10
numero = numero + 5
numero = numero * 2
numero = numero - 4

print(numero)
```

Depois responda no caderno:

1. Qual foi o valor inicial?
2. Quais alterações aconteceram?
3. Qual valor apareceu no `print()`?

Correção da Atividade 2

Vamos acompanhar passo a passo:

```
numero = 10           → número começa com 10
numero = numero + 5   → 10 + 5 = 15
numero = numero * 2   → 15 * 2 = 30
numero = numero - 4   → 30 - 4 = 26
```

Resultado:

```
26
```

Esse acompanhamento manual é chamado de **teste de mesa**.

Tipos de dados

Nem toda informação é igual.

Em programação, precisamos diferenciar os tipos de dados.

Exemplos:

```
"Maria"      → texto
18           → número inteiro
7.5          → número real
True         → valor lógico
```

O tipo do dado influencia o que podemos fazer com ele.

Principais tipos em Python

Tipo	Significado	Exemplo
`str`	texto	`"Ana"`
`int`	número inteiro	`20`
`float`	número com casas decimais	`8.5`
`bool`	verdadeiro ou falso	`True`

No começo, esses quatro tipos já são suficientes para criar muitos programas.

Texto: tipo `str`

Textos ficam entre aspas.

```
nome = "Carlos"
cidade = "Jardim Alegre"
curso = "Lógica de Programação"
```

Podemos usar aspas duplas ou simples:

```
nome = 'Carlos'
```

O importante é abrir e fechar as aspas corretamente.

Número inteiro: tipo `int`

Números inteiros são usados quando não precisamos de casas decimais.

Exemplos:

```
idade = 17
quantidade = 30
ano = 2026
numero_alunos = 22
```

Podemos usar inteiros para:

- contar pessoas;
- contar produtos;
- representar idade;
- representar quantidade.

Número real: tipo `float`

Números reais possuem casas decimais.

Em Python usamos ponto, não vírgula.

```
preco = 19.90
nota = 8.5
altura = 1.75
salario = 2500.00
```

Atenção:

```
preco = 19,90
```

Isso não representa um número decimal do jeito esperado em Python.

Valor lógico: tipo `bool`

O tipo lógico representa apenas duas possibilidades:

```
True
False
```

Exemplos:

```
aprovado = True
maior_de_idade = False
possui_desconto = True
```

Esse tipo será muito importante quando estudarmos condições.

Descobrindo o tipo com `type()`

Python permite verificar o tipo de uma informação.

```
nome = "Ana"
idade = 18
nota = 9.5
aprovado = True

print(type(nome))
print(type(idade))
print(type(nota))
print(type(aprovado))
```

Esse recurso ajuda a entender o que está acontecendo no programa.

Atividade 3 — Classificando dados

No caderno, classifique cada dado como `str`, `int`, `float` ou `bool`.

```
"Senac"
25
3.14
True
"2026"
0
False
"8.5"
8.5
```

Pergunta importante:

> `"8.5"` e `8.5` são a mesma coisa?

Correção da Atividade 3

```
"Senac"  → str
25       → int
3.14     → float
True     → bool
"2026"   → str
0        → int
False    → bool
"8.5"    → str
8.5      → float
```

Atenção:

```
"8.5"
```

é texto, porque está entre aspas.

```
8.5
```

é número real, porque não está entre aspas.

Saída de dados com `print()`

A função `print()` mostra informações na tela.

```
print("Olá, turma!")
```

Também podemos mostrar variáveis:

```
nome = "Ana"
print(nome)
```

E podemos combinar texto com variáveis.

Usando f-string

A f-string facilita mostrar variáveis dentro de textos.

```
nome = "Ana"
idade = 17

print(f"Nome: {nome}")
print(f"Idade: {idade} anos")
```

A letra `f` antes das aspas permite colocar variáveis entre `{ }`.

Isso deixa a saída mais clara e organizada.

Exemplo: ficha de aluno

```
nome = "Ana Paula"
idade = 17
curso = "Lógica de Programação"
cidade = "Ivaiporã"

print("----- FICHA DO ALUNO -----")
print(f"Nome: {nome}")
print(f"Idade: {idade}")
print(f"Curso: {curso}")
print(f"Cidade: {cidade}")
```

Esse exemplo ainda usa dados fixos.

O próximo passo é permitir que o usuário digite os dados.

Entrada de dados com `input()`

A função `input()` permite receber uma informação digitada pelo usuário.

```
nome = input("Digite seu nome: ")

print(f"Olá, {nome}!")
```

O texto dentro do `input()` aparece como mensagem para o usuário.

Aquilo que o usuário digita é guardado na variável.

Exemplo: cadastro simples

```
nome = input("Digite seu nome: ")
cidade = input("Digite sua cidade: ")
curso = input("Digite seu curso: ")

print("----- CADASTRO -----")
print(f"Nome: {nome}")
print(f"Cidade: {cidade}")
print(f"Curso: {curso}")
```

Esse programa já possui:

- entrada;
- armazenamento em variáveis;
- saída de dados.

Atividade 4 — Programa de cadastro

No computador, crie um arquivo chamado:

`cadastro_aluno.py`

O programa deve pedir:

- nome;
- idade;
- cidade;
- curso;
- área da tecnologia que mais interessa.

Depois deve mostrar uma ficha organizada na tela.

Use `input()` e `print()`.

Sugestão para a Atividade 4

```
nome = input("Digite seu nome: ")
idade = input("Digite sua idade: ")
cidade = input("Digite sua cidade: ")
curso = input("Digite seu curso: ")
interesse = input("Área da tecnologia que mais interessa: ")

print()
print("----- FICHA DO ALUNO -----")
print(f"Nome: {nome}")
print(f"Idade: {idade}")
print(f"Cidade: {cidade}")
print(f"Curso: {curso}")
print(f"Interesse: {interesse}")
```

Depois de testar, altere as mensagens para deixar o programa mais apresentável.

Um detalhe importante sobre `input()`

Tudo que vem do `input()` chega como texto.

Mesmo que o usuário digite um número.

Exemplo:

```
idade = input("Digite sua idade: ")

print(type(idade))
```

Se o usuário digitar `18`, o Python recebe como:

```
"18"
```

Ou seja, texto.

Problema: tentando somar textos

Observe:

```
numero1 = input("Digite o primeiro número: ")
numero2 = input("Digite o segundo número: ")

soma = numero1 + numero2

print(soma)
```

Se o usuário digitar `10` e `5`, o resultado será:

```
105
```

Por quê?

Porque o Python juntou dois textos.

Conversão de tipos

Para fazer cálculo, precisamos converter o texto para número.

Principais conversões:

```
int()
float()
str()
```

Exemplo:

```
idade = int(input("Digite sua idade: "))
```

Agora a idade será tratada como número inteiro.

Soma correta com `int()`

```
numero1 = int(input("Digite o primeiro número: "))
numero2 = int(input("Digite o segundo número: "))

soma = numero1 + numero2

print(f"Resultado: {soma}")
```

Agora, se o usuário digitar `10` e `5`, o resultado será:

```
15
```

Quando usar `int()` e `float()`?

Use `int()` quando o valor for inteiro:

```
idade = int(input("Digite sua idade: "))
quantidade = int(input("Digite a quantidade: "))
```

Use `float()` quando o valor puder ter casas decimais:

```
preco = float(input("Digite o preço: "))
nota = float(input("Digite a nota: "))
altura = float(input("Digite a altura: "))
```

Atividade 5 — Calculadora de soma

Crie um arquivo chamado:

`soma.py`

O programa deve:

1. pedir dois números inteiros;
2. somar os dois valores;
3. mostrar o resultado.

Depois, altere o programa para mostrar também:

- subtração;
- multiplicação;
- divisão.

Exemplo da Atividade 5

```
numero1 = int(input("Digite o primeiro número: "))
numero2 = int(input("Digite o segundo número: "))

soma = numero1 + numero2
subtracao = numero1 - numero2
multiplicacao = numero1 * numero2
divisao = numero1 / numero2

print(f"Soma: {soma}")
print(f"Subtração: {subtracao}")
print(f"Multiplicação: {multiplicacao}")
print(f"Divisão: {divisao}")
```

Atenção:

> O que acontece se o segundo número for zero?

Guardem essa pergunta para quando estudarmos condições.

Operadores aritméticos

Operador	Significado	Exemplo
`+`	adição	`10 + 2`
`-`	subtração	`10 - 2`
`*`	multiplicação	`10 * 2`
`/`	divisão	`10 / 2`
`//`	divisão inteira	`10 // 3`
`%`	resto da divisão	`10 % 3`
`**`	potência	`2 ** 3`

Precedência de operadores

Python respeita uma ordem de cálculo.

```
resultado = 10 + 2 * 3
print(resultado)
```

O resultado será 16, porque a multiplicação vem antes da soma.

Se quisermos mudar a ordem, usamos parênteses:

```
resultado = (10 + 2) * 3
print(resultado)
```

Agora o resultado será 36.

Atividade 6 — Calculando média

Crie um arquivo chamado:

`media.py`

O programa deve:

1. pedir o nome do aluno;
2. pedir três notas;
3. calcular a média;
4. mostrar o nome do aluno e a média final.

Fórmula:

```
media = (nota1 + nota2 + nota3) / 3
```

Exemplo da Atividade 6

```
nome = input("Digite o nome do aluno: ")
nota1 = float(input("Digite a primeira nota: "))
nota2 = float(input("Digite a segunda nota: "))
nota3 = float(input("Digite a terceira nota: "))

media = (nota1 + nota2 + nota3) / 3

print("----- RESULTADO -----")
print(f"Aluno: {nome}")
print(f"Média: {media}")
```

Depois podemos melhorar a saída usando formatação.

Formatando casas decimais

Às vezes o resultado aparece com muitas casas decimais.

Exemplo:

```
media = 7.6666666667
```

Podemos formatar com duas casas decimais:

```
print(f"Média: {media:.2f}")
```

O resultado será:

```
Média: 7.67
```

Atividade 7 — Loja simples

Crie um arquivo chamado:

`loja.py`

O programa deve pedir:

- nome do produto;
- preço unitário;
- quantidade comprada.

Depois deve calcular:

```
total = preco * quantidade
```

E mostrar um resumo da compra.

Exemplo da Atividade 7

```
produto = input("Digite o nome do produto: ")
preco = float(input("Digite o preço unitário: R$ "))
quantidade = int(input("Digite a quantidade: "))

total = preco * quantidade

print("----- RESUMO DA COMPRA -----")
print(f"Produto: {produto}")
print(f"Preço unitário: R$ {preco:.2f}")
print(f"Quantidade: {quantidade}")
print(f"Total: R$ {total:.2f}")
```

Esse exercício junta tudo que vimos até aqui.

Erros comuns nesta aula

Alguns erros que podem aparecer:

1. esquecer aspas em textos;
2. esquecer parênteses no `print()`;
3. usar vírgula em número decimal;
4. tentar calcular com valor vindo do `input()` sem converter;
5. escrever nome de variável com espaço;
6. confundir `=` com igualdade matemática.

Em programação, errar faz parte do processo.

O importante é aprender a ler o erro e corrigir.

Onde estamos no curso

Nesta aula, ainda estamos na base da lógica de programação.

A sequência até aqui está assim:

```
Aula 01 → algoritmo, entrada, processamento e saída
Aula 02 → variáveis, tipos, entrada, saída e cálculos
Próxima aula → tomada de decisão com condições
```

Ou seja: antes de decidir, repetir ou usar listas, precisamos saber **guardar dados**, **receber informações** e **fazer cálculos simples**.

Mini revisão

Hoje vimos:

- variável;
- atribuição;
- tipos de dados;
- `str`, `int`, `float` e `bool`;
- saída com `print()`;
- entrada com `input()`;
- conversão com `int()` e `float()`;
- operadores aritméticos;
- formatação com f-string;
- primeiros cálculos em Python.

Desafio final da aula

Crie um programa chamado:

`viagem.py`

O programa deve pedir:

- nome do motorista;
- distância da viagem em km;
- consumo médio do veículo em km/l;
- preço do litro do combustível.

Depois deve calcular:

```
litros = distancia / consumo
custo = litros * preco_litro
```

E mostrar um resumo da viagem.

Exemplo de saída esperada

```
----- RESUMO DA VIAGEM -----
Motorista: Dieimes
Distância: 120.0 km
Consumo médio: 12.0 km/l
Litros necessários: 10.00 litros
Custo estimado: R$ 58.90
```

Pergunta para pensar:

> E se o usuário digitar consumo igual a zero?

Na próxima aula, vamos começar a tomar decisões no código com estruturas condicionais.